Project Iteration 1.1: Detailed User Stories, Requirements, and Initial Prototype
Project Name: PeerPulse
Team Number: 7
Team Members: Aditya Rajesh, Vardan Abrol, Cheuk Yin Maurice Yu and Mordecai Dada

1. Document Overview:

This document outlines the deliverables for Iteration 2 of the project, focusing on defining detailed user stories, specifying functional and non-functional requirements, and developing an initial prototype. This iteration also emphasizes proper documentation and planning, which sets the stage for subsequent development phases. The goal is to establish a solid foundation by clarifying what the system should do and beginning the implementation with a functional prototype.

2. Functional and Non-functional Requirements

2.1. Functional Requirements

The following functional requirements define the core features that the system must provide to support mentors, mentees, and administrators.

- **Requirement 1**: User Registration and Authentication. The system shall allow mentors and mentees to register and log in to their accounts. Mentor registrations shall require administrator approval before account activation, while mentees can access available mentoring sessions after registration.

- **Requirement 2:** User Profile Management. The system shall allow mentors and mentees to create and update their profiles, including personal information, academic year, major, completed or ongoing courses, certifications, and technical skills.

- **Requirement 3:** Course Repository Management. The system shall allow administrators to add and delete courses, including course codes and descriptions. Mentors shall be able to register for or withdraw from courses they wish to provide mentorship for.

- **Requirement 4:** Mentor–Mentee Search and Matching. The system shall enable users to search for potential mentors or mentees based on course codes, course titles, and technical skills. It shall display matching profiles and notify users when no matching results are found.

- **Requirement 5:** Mentorship Session Scheduling. The system shall provide a calendar or timesheet interface that allows mentors to create and manage available mentorship sessions. Mentees shall be able to view available sessions, register for sessions, and cancel their registrations. All scheduling information shall be stored and updated accordingly.

- **Requirement 6:** Session Feedback. The system shall allow mentors to provide feedback to mentees after mentorship sessions. Mentees shall be able to access and review their session feedback.

- **Requirement 7:** Role-Based Access Control. The system shall provide different levels of access for administrators, mentors, and mentees, ensuring users can only access features and information authorized for their respective roles.

2.2. Non-Functional Requirements

The following non-functional requirements define the quality attributes and performance expectations necessary to ensure the system operates effectively.

- **Usability:** The system should provide an intuitive and user-friendly interface that allows users with different levels of technical experience to navigate the platform easily. Essential features, such as profile management, mentor searching, and session booking, should be accessible with minimal steps and clear instructions.

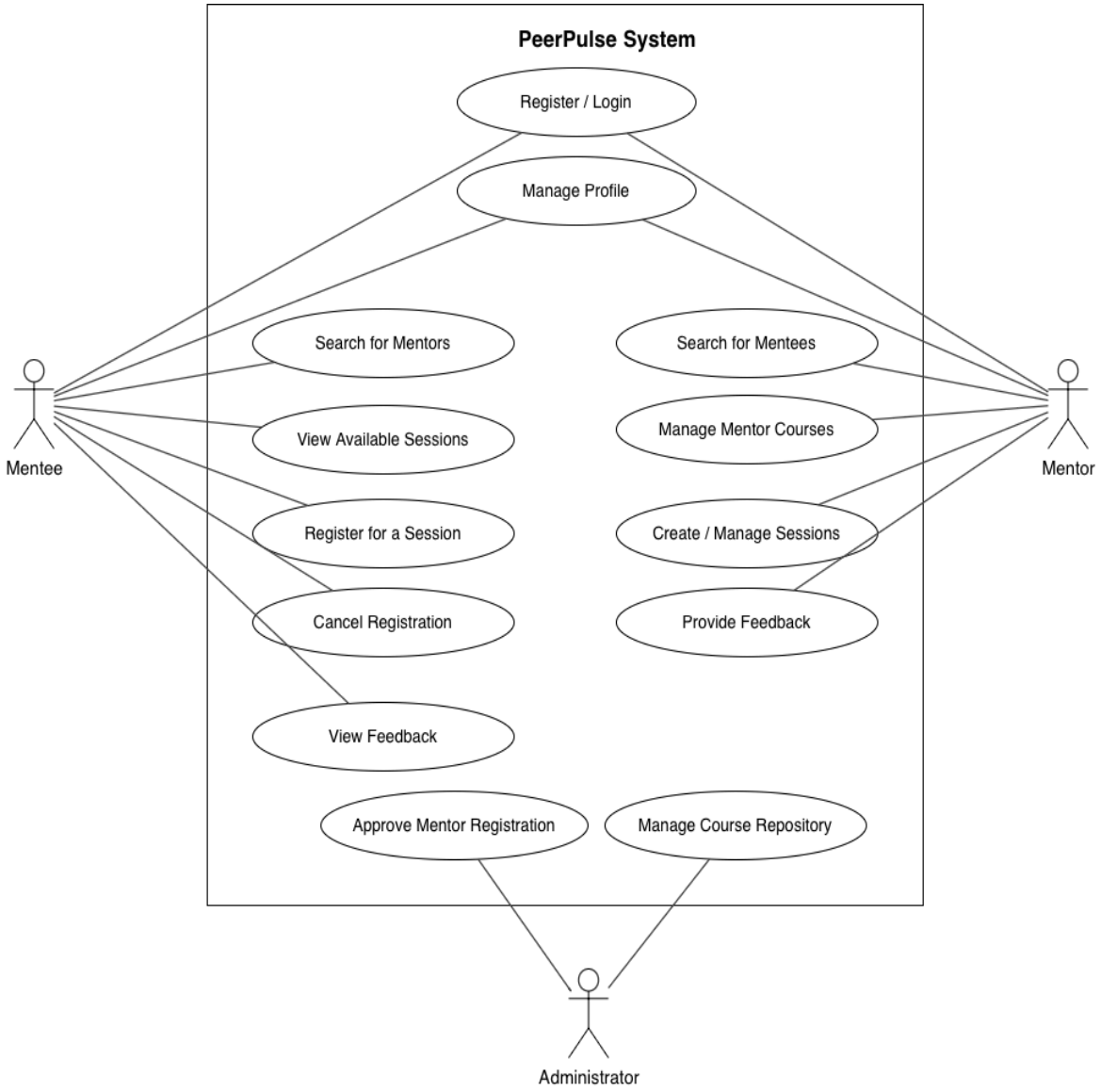
- **Scalability:** The system should be designed to accommodate an increasing number of users, mentorship sessions, and course records without significantly affecting its performance. The architecture should also support the future implementation of additional features without requiring major system modifications.

- **Security:** The system should protect users' personally identifiable information (PII) through secure authentication, role-based access control, and appropriate data protection measures. Mentor and mentee profile information should be stored separately or logically segregated to prevent unauthorized access and maintain user privacy.

- **Performance:** The system should provide fast and reliable responses when performing essential operations, including searching for mentors, updating profiles, and booking sessions. As a proposed benchmark, common operations should respond within 3 seconds under normal usage conditions to ensure a smooth user experience.

3. Use Cases

3.1 Use Case Diagram



3.2 Use Case Descriptions

- **Use Case 1**: User registration and Login
  Actors: Mentor, Mentee
  Description: Users can register and log in to PeerPulse.
  Preconditions: The user must not already have an account when registering. An active account is required to log in.
  Main Flow:
  1. The user opens PeerPulse.
  2. The user registers for an account or enters their login details.

3. The system checks the information.
4. The user logs in if their account is active.

Postconditions: The user can access their account. Mentor accounts require administrator approval before activation.

- **Use Case 2**: Manage User Profile
  Actors: Mentor, Mentee
  Description: Users can add or update their profile information.
  Preconditions: The user must be logged in.
  Main Flow:
  1. The user opens their profile.
  2. The user adds or changes their information, courses, and skills.
  3. The user saves the changes.
  4. The system updates the profile.

  Postconditions: The profile information is saved.

- **Use Case 3**: Manage Courses
  Actors: Administrator
  Description: Administrators can add or remove courses.
  Preconditions: The user must be logged in.

  Main Flow:
  1. The administrator opens the course repository.
  2. The administrator adds a course or removes an existing course.
  3. The system saves the changes.

  Postconditions: The course repository is updated.

- **Use Case 4**: Manage Mentor Courses
  Actors: Mentor
  Description: Mentors can select or remove courses they want to provide mentorship for.
  Preconditions: The mentor must be logged in.

  Main Flow:
  1. The mentor opens the available courses.
  2. The mentor selects or removes courses.
  3. The system saves the changes.

  Postconditions: The mentor course list is updated.

- **Use Case 5**: Search for Mentors/Mentees
  Actors: Mentor, Mentee
  Description: Users can search for mentors or mentees based on courses and skills.

Preconditions: The user must be logged in.

Main Flow:

1. The user opens the search page.
2. The user enters a course code, course name, or skill.
3. The system displays matching profiles or a no-results message.
4. The user selects a profile to view, if available.

Postconditions: The user can view any matching profiles.

- **Use Case 6**: Manage Mentorship Sessions
  Actors: Mentor, Mentee
  Description: Mentors can create and manage available sessions. Mentees can view, register for or cancel sessions.
  Preconditions: The user must be logged in.

  Main Flow:

  1. The mentor creates a session by selecting a date and time.
  2. The system displays the available session.
  3. The mentee selects a session and registers for it.
  4. The system saves the registration and updates the session details.

  Postconditions: Session details are updated. Mentees can also cancel their registrations.

- **Use Case 7**: Session Feedback
  Actors: Mentor, Mentee
  Description: Mentors give feedback after sessions, and mentees can view it.
  Preconditions: A mentorship session must be completed before feedback can be submitted.
  Main Flow:

  1. The mentor opens a completed session.
  2. The mentor enters feedback for the mentee.
  3. The system saves the feedback.
  4. The mentee opens the feedback section to read it.

  Postconditions: The feedback is saved and available to the mentee.

- **Use Case 8**: Approve Mentor Registration
  Actors: Administrator
  Description: Administrators review and approve or reject mentor registrations.
  Preconditions: A mentor has submitted a registration request.
  Main Flow:

  1. The administrator opens the pending registrations.
  2. The administrator reviews the mentor information.
  3. The administrator approves or rejects the request.

4. The system updates the registration status.

Postconditions: The mentor account is approved and activated or rejected.

4. Detailed user Stories

4.1 Category 1: Sign up

- User story 1: Mentor Sign-up
  - As a mentor, I want to be able to apply to become a mentor by providing details like my name, contact information, certifications, previous courses taken (Transcript), mentor topic selection (What subject/course I want to mentor on) so that I can apply and provide all the necessary information for the administrators to make a decision on whether I am allowed to be a mentor.
  - Acceptance criteria:
    - The sign up system will prompt the user to choose a selection between mentor and mentee where the user will select mentor
    - Upon selecting a mentor, questions related to becoming a mentor will appear and the user must fill them out
    - The submitted response will be saved and sent to the administrators where they will have the choice on allowing the user to join the mentor team
    - If the mentor is allowed to continue, their account will be made using the information they provided and will be allowed to access the mentor related tools and information
    - The log in details will also be saved for future use, they will be asked to set up their own password however once the account is created in order for them to access the details
  - Priority:
    - High

- User story 2: Mentee Sign-up
  - As a mentee, I want to be able to sign up to the program by providing details like my name, contact information, certifications, previous courses taken (Transcript), username, password so that I can get my profile created and get access to the mentee side of the platform.
  - Acceptance criteria:
    - The sign up system will prompt the user to choose a selection between mentor and mentee where the user will select mentee
    - Upon selecting a mentor, questions related to becoming a mentee will appear and the user must fill them out
    - They will be prompted to make a username and password as well that will be linked with their email
    - The submitted response will be saved and the account will be created and given access to the platform
  - Priority:

        - High

- User story 3: Administrator Sign-in
    - As an administrator, I will need to log in with my account in order for me to perform the required moderation, auditing and management of the platform.
    - Acceptance criteria:
        - The sign up system will also have an administrator log in where admin credentials will be used, these will be pre-made
        - Once log in with the admin credentials is complete, the administrator will have access to moderate, audit and manage the platform.
    - Priority:
        - High

4.1 Category 2: Platform usage

- User story 1: Mentor platform usage
    - As a mentor, I want to be able to post my times where I will be available for certain subjects and the methods at which I will be able to meet. I want to be able to also give feedback to my mentees in order to help them understand and grow.
    - Acceptance criteria:
        - The mentor side of the application will have access to set times and dates on when a mentor will be available to teach courses so a date and time selection must be added. This data will also be stored to make it available for mentees and administrators as well
        - The mentors will also have access to give feedback to their mentees via the platform so that it can be audited if needed and will allow easier communication for growth
        - Will have access to a report button in the event they need to report something to an admin
    - Priority:
        - High

- User story 2: Mentee platform usage
    - As a mentee, I want to be able to view what times and what instructors are available and to sign up for mentoring slots as needed. I need to be able to view the feedback as well on the site to be able to improve on my skills as highlighted by my mentor. I want to also be able to filter through mentors to see which mentor best aligns with what I am looking for.
    - Acceptance criteria:
        - The mentee side of the platform will have the option to sign up for mentoring sessions on the date/time selection
        - The mentee side of the platform will have the option to view feedback for a particular session
        - The mentee side of the platform will have the option to search for specific mentors teaching in order for them to match with the best mentor based on their needs

        - Will have access to a report button in the event they need to report something to an admin
    - Priority:
        - High

- User story 3: Administrator Platform Usage
    - As an administrator, I will need to perform the required moderation, auditing and management of the platform. I will also be accepting applications for mentors and giving them the account activation so that they may start mentoring.
    - Acceptance criteria:
        - The administrator side of the platform will have the options to view both mentor and mentee sides of the platform, the will be able to ensure both parties are following the guidelines and can view the feedback page to audit whether everything is going smoothly
        - They will receive the reports from both the mentor and mentee via mail from the platform and will be able to take action as needed
        - They will have permissions to remove mentors and mentees from the platform should the need arise
    - Priority:
        - High

5. Initial Prototype Guideline

5.1 Prototype Features

The initial PeerPulse prototype is a set of PDF wireframes illustrating the proposed screens and core user interactions. **The wireframe PDF has been uploaded to the team's GitHub repository alongside the project documentation.**

The wireframes are organized by user role:

- **Mentee:** Dashboard, academic profile, mentor search, available sessions, session registration and cancellation, feedback viewing, and course browsing.
- **Mentor:** Dashboard, academic profile, mentee search, availability management, session creation and editing, feedback submission, and selection of mentoring subjects.
- **Administrator:** Dashboard, mentor application approval or rejection, course creation and deletion, user management, feedback auditing, and report review.
- **Shared access:** Login and mentor or mentee registration, including transcript information for mentor applications.

The screens use simple boxes, labels, and placeholders to show the layout and intended functionality. Internal PDF links allow navigation between selected screens, demonstrating the proposed user journeys.

This prototype represents the interface design rather than an implemented system. Forms do not submit data, and authentication, database storage, searching, scheduling, and administrator actions are not operational. These capabilities will be developed and tested in subsequent iterations.

5.2 Development Approach

The team will use an **Agile development** approach to turn the PeerPulse wireframes into a working system. Features will be developed in small increments, reviewed regularly, and refined based on testing and user feedback. Core functionality will take priority over visual polish.

Development will begin with the application structure and navigation for mentees, mentors, and administrators. Shared components, such as profile forms, search fields, and calendars, will be reused, while permissions will control the actions available to each role.

Preliminary data storage will support accounts, profiles, courses, sessions, and feedback. Mock data may be used initially to test screen interactions before connecting them to a database.

Features will then be implemented in the following order:

1. **Registration, login, and profiles:** Create accounts, manage academic information, and support administrator approval of mentor applications.
2. **Course management and searching:** Maintain course records, allow mentors to select mentoring subjects, and search for peers by course or technical skill.
3. **Availability and scheduling:** Allow mentors to publish sessions and mentees to register or cancel, with checks to prevent scheduling conflicts.
4. **Feedback and administration:** Enable mentors to submit session feedback, mentees to view it, and administrators to review applications, users, and reports.

The code will follow a modular, object-oriented structure, separating account management, profiles, courses, searching, sessions, feedback, and administration. Each feature will be built, tested against its acceptance criteria, and refined before integration with the wider system.

GitHub will track developer tasks, regular commits, reviews, and outstanding work. Repository documentation will record implemented features, setup instructions, testing results, and known limitations.

5.3 Task Completion and Team Collaboration

The team will break the user stories into manageable developer tasks. Each task will have an assigned owner, acceptance criteria, priority, and estimated time effort. The following preliminary estimates will be reviewed during Agile iteration planning.

Developer task	**Time Estimate**
Set up the testing environment	Week 6
Set up the application and role-specific navigation	Week 6
Set up preliminary data storage and mock data	Week 6

Implement registration, login, and role permissions including unit tests	Week 6-7
Implement profile creation and updating including unit tests	Week 6-7
Implement course management and mentor subject selection including unit tests	Week 7
Implement peer searching filtering and availability matching including unit tests	Week 7-8
Implement session creation, registration and cancellation including unit tests	Week 7-8
Implement feedback submission and viewing including unit tests	Week 8
Implement mentor approval and administrator management including unit tests	Week 8-9
Implement reporting and report review including unit tests	Week 9
Connect features, test user journeys, and fix bugs.	Week 8-9; repeat after improvement in week 10-12
Update repository documentation	Week 6-12; alongside development

Tasks will be distributed among the four team members according to availability and dependencies. A shared GitHub Projects **board** has been created to be used to track progress through **To Do, In Progress, In Review, and Done.**

Team members will commit frequently with clear messages and develop features in separate branches. Pull requests will allow another team member to review changes before integration into development. Tested, stable versions will be merged into main.

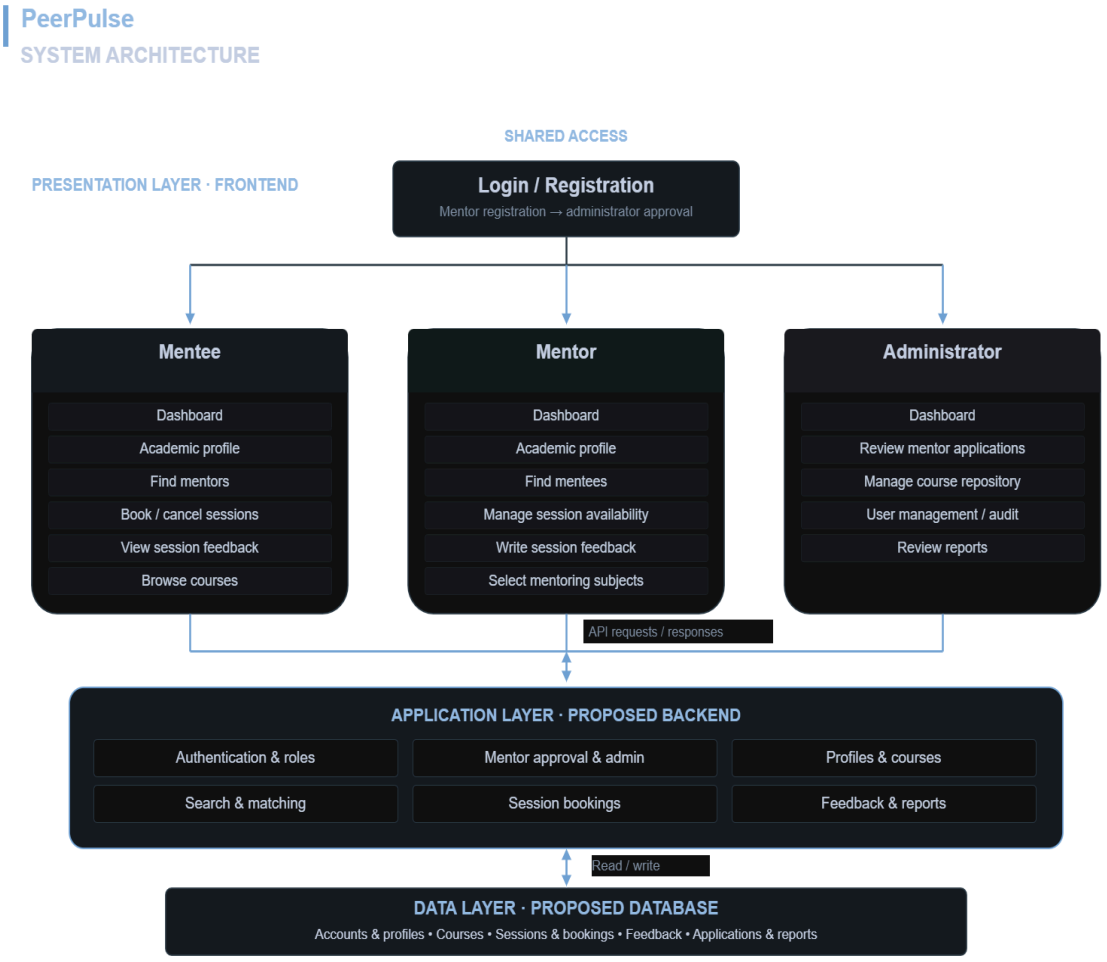
A task will be considered complete when its acceptance criteria are met, relevant tests pass, review feedback is addressed and documentation is updated. The wireframes will guide implementation, while the requirements will define the expected behavior.

Regular Agile check-ins will help the team discuss progress, resolve blockers, and coordinate related features. At the end of each iteration, the team will review completed work and update the backlog using testing results and feedback.

The current wireframe PDF and project documentation are stored in GitHub. Implementation progress, known limitations, and outstanding tasks will continue to be documented in the repository.

6. Documentation Updates

6.1. System Architecture Wireframe diagram



The system architecture's three main components:

- **User Interface (Frontend):** Provides role-specific dashboards and navigation pages. Mentees can manage academic profiles, search for mentors, book sessions, and view feedback. Mentors can manage their profiles, search for mentees, publish session availability, and provide feedback. Administrators can review mentor applications, manage courses and users, and review reports.
- **Application Logic (Backend):** The proposed backend will handle the system's core operations, including user authentication, role-based access control, mentor approval, profile management, search and filtering, session scheduling, and feedback processing.
- **Data Storage (Database):** The proposed database will store information such as user accounts, academic profiles, course records, session availability, bookings, feedback, and administrative records.

Component Interaction:

Users will interact with the platform through the frontend interfaces. When an action is performed, such as searching for a mentor or registering for a session, the frontend will send a request to the backend. The backend will process the request and retrieve or update relevant information in the database before returning the result to the user interface.

For example, mentors will be able to publish available mentorship sessions, which mentees can view and register for through their calendars. Administrators will manage mentor account approvals and course information through a separate dashboard.

The current prototype remains at the wireframe stage. Therefore, the architecture represents the intended system structure, and the backend implementation, database design, and technology stack have not yet been finalized.

6.2. Planning Document and Log File

All planning document and log file (including prototype sketches), are saved in a github folder called "Project plan and prototype"

7. Iteration Plan and Backlog

7.1 Iteration plan

- Week 3: Finalize functional requirements and begin initial prototype development.
- Week 4: Complete the initial prototype and refine key user stories.
- Week 5: Review and update the use cases, user stories, and prototype based on feedback.

7.2 Updated backlog

- Feature: Sign up (Mentor)
  - Description: Allow mentors to apply with credential, contact information and transcript and to get admin approval before account creation
  - Priority: High
- Feature: Sign up (Mentee)
  - Description: Allow mentees to sign up for the platform by creating their profile and setting up credentials
  - Priority: High
- Feature: Admin Login
  - Description: Allow administrators to log in using admin credentials
  - Priority: High
- Feature: Mentor Platform Set up
  - Description: Set up the platform for the mentors to view all required items pertaining to the mentors requirements

- ➢ Priority: High

- ❖ Feature: Mentee Platform Set up
  - ➢ Description: Set up the platform for the mentors to view all required items pertaining to the mentee's requirements
  - ➢ Priority: High
- ❖ Feature: Administrator Platform Set up
  - ➢ Description: Set up the platform for the mentors to view all required items pertaining to the admin's requirements
  - ➢ Priority: High
- ❖ Feature: Reporting
  - ➢ Description: Allow for reporting for both mentor and mentee and link it to the admin email in order for admin's to get the report and act
  - ➢ Priority: Medium
- ❖ Feature: UI basics
  - ➢ Description: Basic UI to cover the back end processes and make the website more user friendly
  - ➢ Priority: Medium
- ❖ Feature: Set up data storage
  - ➢ Description: Set up a data storage to store all the user data and allow for access as needed
  - ➢ Priority: High